

# Draft the API Specification

## 1. Overview

After clarifying requirements, the next step is:

Define how users and systems interact with your service.

That means designing:

- REST APIs
- Request/Response contracts
- CRUD operations
- Authentication flows

## 2. Why API Design Matters in Interviews

Good API design shows:

- Clear thinking
- Practical engineering sense
- Understanding of system boundaries
- User workflows understanding

## 3. Time Management

Spend:

- **3–5 minutes max**

Goal is to **draft quickly**, not over-engineer.

## 4. What to Define

For each endpoint, define:

- HTTP Method (**GET**, **POST**, etc.)
- URL path
- Query parameters
- Request body
- Response body
- Auth requirement

## Common REST Methods

| Method | Purpose        |
|--------|----------------|
| GET    | Read data      |
| POST   | Create data    |
| PUT    | Full update    |
| PATCH  | Partial update |
| DELETE | Remove data    |

## 5. Generic API Flow

None

Client → API Gateway → Service → DB

## 6. Common APIs in Most Systems

### Health Check

None

GET /health

#### Purpose:

- Used by load balancers / monitoring systems

#### Response:

JSON

```
{  
  "status": "healthy"  
}
```

## Signup

None

POST /signup

### Request:

JSON

```
{  
  "email": "user@mail.com",  
  "password": "securePass"  
}
```

## Login

None

POST /login

### Response:

JSON

```
{  
  "token": "jwt_token"  
}
```

## 7. User Management APIs

### Get User

None

GET /users/{userId}

### Update User

None

PUT /users/{userId}

### Delete User

None

DELETE /users/{userId}

## 8. Content APIs (Example: Social App)

### Create Post

None

POST /posts

JSON

```
{  
  "text": "Best bagel in town!"  
}
```

## Fetch Feed

None

```
GET /posts?limit=20&cursor=abc123
```

## Delete Post

None

```
DELETE /posts/{postId}
```

# 9. Moderation APIs

## Report Content

None

```
POST /posts/{postId}/report
```

Reason:

- Spam
- Abuse
- Illegal content

## 10. Search APIs

None

```
GET /search?q=bagel&location=Surat
```

## 11. Pagination

Never return unlimited data.

Use:

### Offset Pagination

None

```
GET /posts?page=2&limit=20
```

### Cursor Pagination Better at scale

None

```
GET /posts?cursor=xyz
```

## 12. Authentication & Security

Use:

- JWT tokens
- OAuth2
- API keys (B2B systems)

Header:

None

Authorization: Bearer <token>

## 13. Good API Design Principles

Use nouns not verbs

✓ /users/123/posts

✗ /getUserPosts

Version APIs

None

GET /v1/posts

Proper Status Codes

| Code | Meaning |
|------|---------|
|------|---------|



|     |              |
|-----|--------------|
| 200 | Success      |
| 201 | Created      |
| 400 | Bad Request  |
| 401 | Unauthorized |
| 404 | Not Found    |
| 500 | Server Error |

## 14. Interview Strategy

Say:

“I’ll quickly define a few key APIs to represent system interactions, then move to architecture since time is limited.”

This shows maturity.

## 15. Example: Food Delivery APIs

None

GET /restaurants?location=Surat

POST /orders

GET /orders/{id}

PATCH /orders/{id}/cancel

## 16. What Interviewer Wants Here

Not perfect syntax.

They want to know:

- Can you map features → APIs?
- Can you think practically?
- Do you understand contracts?

## 17. Final Summary

- Draft APIs quickly (3–5 min)
- Cover core workflows only
- Mention auth + pagination
- Use REST conventions
- Move quickly to architecture

## Pro Tip (Very Important)

If system is internal/high performance:

Mention:

- gRPC instead of REST

That's a strong signal.